

Game Collection API

API Documentation — v1.0.0

GitHub: github.com/CyrDuck/game-collection-api

1. Overview

The Game Collection API is a RESTful web service built with FastAPI and SQLite that allows users to manage a personal video game library. It supports full CRUD operations on game entries, filtering by platform and status, full-text title search, and analytics endpoints for genre and platform breakdowns. The API ships with a seed dataset of 50 games across PC, PS5, Switch, and Xbox platforms.

Base URL	http://127.0.0.1:8000
Framework	FastAPI 0.115.0 (Python 3.13)
Database	SQLite via SQLAlchemy ORM
Response Format	JSON
Interactive Docs	http://127.0.0.1:8000/docs (Swagger UI)
GitHub Repository	https://github.com/CyrDuck/game-collection-api

2. Game Data Model

Every game entry contains the following fields:

Field	Type	Required	Constraints	Example
id	integer	auto	Primary key, auto-increment	1
title	string	Yes	Max 200 characters	"Elden Ring"
platform	string	Yes	Max 50 characters	"PC"
genre	string	No	Optional	"RPG"
release_year	integer	No	Range: 1970–2030	2022
status	string	No	wishlist / playing / completed / dropped	"completed"
rating	float	No	Range: 1.0–10.0	9.5
notes	string	No	Free text	"Masterpiece"
created_at	datetime	auto	Set on creation	"2026-04-25T10:00:00"
updated_at	datetime	auto	Updated on edit	null

3. Endpoints Summary

Method	Endpoint	Description
GET	/	API information and version
GET	/health	Server health check

GET	/stats	Collection statistics (total, by status)
GET	/stats/genres	Game count grouped by genre
GET	/stats/platforms	Game count grouped by platform
POST	/games/	Create a new game entry
GET	/games/	List all games with optional filters
GET	/games/{id}	Get a specific game by ID
PUT	/games/{id}	Update a game entry (partial update supported)
DELETE	/games/{id}	Delete a game entry
GET	/games/search/title	Search games by title keyword

4. Endpoint Details

4.1 GET / — API Info

Returns basic API metadata.

Response (200):

```
{ "message": "Game Collection API", "version": "0.2.0", "docs": "/docs" }
```

4.2 GET /health — Health Check

Returns server health status. Useful for uptime monitoring.

Response (200):

```
{ "status": "ok" }
```

4.3 GET /stats — Collection Statistics

Returns aggregate counts of games grouped by play status.

Response (200):

```
{ "total_games": 50, "completed": 24, "playing": 9, "wishlist": 10 }
```

4.4 GET /stats/genres — Genre Analytics

Returns game counts grouped by genre, ordered by count descending.

Response (200):

```
{ "genre_breakdown": [ { "genre": "RPG", "count": 8 }, { "genre": "Action RPG", "count": 7 }, { "genre": "Action Adventure", "count": 6 } ] }
```

4.5 GET /stats/platforms — Platform Analytics

Returns game counts grouped by platform, ordered by count descending.

Response (200):

```
{ "platform_breakdown": [ {"platform": "PC", "count": 30}, {"platform": "PS5", "count": 10}, {"platform": "Switch", "count": 7} ] }
```

4.6 POST /games/ — Create Game

Creates a new game entry. title and platform are required; all other fields are optional.

Request Body:

```
{ "title": "Elden Ring", // required, string "platform": "PC", // required, string "genre": "RPG", // optional "release_year": 2022, // optional, integer 1970-2030 "status": "completed", // optional: wishlist/playing/completed/dropped "rating": 9.5, // optional, float 1.0-10.0 "notes": "Masterpiece" // optional, string }
```

Response (201 Created):

```
{ "id": 1, "title": "Elden Ring", "platform": "PC", "genre": "RPG", "release_year": 2022, "status": "completed", "rating": 9.5, "notes": "Masterpiece", "created_at": "2026-04-25T10:00:00", "updated_at": null }
```

4.7 GET /games/ — List All Games

Returns all games. Supports optional filtering by platform and status.

Query Parameters:

Parameter	Type	Required	Description
platform	string	No	Case-insensitive partial match (e.g. "PC")
status	string	No	Exact match: wishlist / playing / completed / dropped

Example: GET /games/?platform=PC&status=completed

Response (200): Array of Game objects matching the filter.

4.8 GET /games/{game_id} — Get Single Game

Returns a single game by its integer ID.

Example: GET /games/1

Response (200): Single Game object. Returns 404 if ID does not exist.

4.9 PUT /games/{game_id} — Update Game

Updates an existing game. Supports partial updates — only fields included in the request body are changed. Fields not included remain unchanged.

Example Request Body (partial update):

```
{"status": "completed", "rating": 9.0}
```

Response (200): Full updated Game object. Returns 404 if ID not found.

4.10 DELETE /games/{game_id} — Delete Game

Permanently removes a game entry from the database.

Response: 204 No Content on success. Returns 404 if ID not found.

4.11 GET /games/search/title — Search by Title

Performs a case-insensitive keyword search across all game titles.

Query Parameters:

Parameter	Type	Required	Description
q	string	Yes	Search keyword (case-insensitive, partial match)

Example: GET /games/search/title?q=ring

Response (200): Array of matching Game objects (e.g. returns "Elden Ring").

Returns 400 if query parameter q is empty.

5. Error Codes

Status Code	Meaning	When it occurs
200 OK	Success	GET and PUT requests succeed
201 Created	Resource created	POST /games/ succeeds
204 No Content	Deleted successfully	DELETE succeeds
400 Bad Request	Invalid input	Empty search query
404 Not Found	Resource missing	Game ID does not exist
422 Unprocessable Entity	Validation failed	Invalid status value, rating out of range
500 Internal Server Error	Server error	Unexpected database failure

All error responses follow this format:

```
{"detail": "Game not found"}
```

6. Running the API Locally

```
git clone https://github.com/CyrDuck/game-collection-api.git cd game-collection-api
python -m venv venv venv\Scripts\activate pip install -r requirements.txt uvicorn
main:app --reload python seed_data.py # optional: load 50 sample games
```

Then open **<http://127.0.0.1:8000/docs>** for the interactive Swagger UI.